

A4：词义消歧与语义角色标注系统

1. prompt

- 接下来我将进行《自然语言处理》这门课的A2作业：Week 5 随堂 Vibe 实验：词义消歧与语义角色标注系统。在 Week 5 的理论课中，我探讨了语义分析的两个核心任务：
 1. 词义消歧 (WSD)：解决一词多义问题，方法从传统的基于词典释义匹配，发展到基于上下文向量表示的深度学习模型。
 2. 语义角色标注 (SRL)：提取句子中的“谓词-论元”结构（即“谁对谁在什么时间地点做了什么”），揭示句子的浅层语义框架。
- 因此，本次实验我需要利用AI 辅助编程工具，使用 Python 的 Streamlit 框架，结合 nltk、spaCy 和 transformers 等自然语言处理库，构建一个多标签页、包含 WSD 和 SRL 两个功能模块的交互式 Web 分析系统。通过真实代码的运行，对比传统方法与深度学习方法在处理复杂语义时的差异。本次实验分为2个模块，我将分模块发送。（请了解这个作业的任务背景，不需要回复我任何信息）

Copilot & GPT-5.2-Codex:

已了解你的作业背景与目标，后续你按模块发我就行。

- 模块 1：词义消歧 (WSD) 对比测试（对应基于释义匹配的消歧与基于上下文向量/BERT的消歧）——在 Streamlit 应用的第一个标签页中，实现一个词义消歧 (WSD) 模块。
 1. 提供两个输入框：第一个输入包含多义词的句子（如 'I went to the bank to deposit my money.'），第二个输入框指定目标多义词（如 'bank'）。
 2. 传统方法实现：使用 nltk.wsd 中的 lesk 算法。输出 Lesk 算法预测的 WordNet 词义 (Synset) 及其具体的英文定义 (Definition)。
 3. 上下文向量表示：使用 Hugging Face 的 transformers 库加载 bert-base-uncased 模型。提取目标词在该句子中的上下文词向量 (Contextual Embedding)。
 4. 对比验证：让用户再输入第二个句子（如 'I sat by the river bank.'）。提取这第二个句子中目标词的 BERT 词向量。计算并输出这两个 BERT 词向量的余弦相似度。

Copilot & GPT-5.2-Codex:

你这个模块需求我理解为：先把 Streamlit 应用里“第一个标签页（WSD 对比测试）”完整落地，包括 Lesk、BERT 向量提取和双句余弦相似度。先快速查看当前工作区里是否已有代码可复用，然后我直接实现可运行版本。

已完成模块 1 的 Streamlit 实现，并已通过静态错误检查（无报错）。

实现内容对应你的 4 点要求：

1. 两个核心输入 + 对比句输入

- 第一句（含多义词）
- 目标词
- 第二句（用于向量对比）

2. 传统方法（Lesk）

- 使用 `nltk.wsd.lesk` 预测 Synset
- 输出 Synset 名称与英文 Definition

3. 上下文向量（BERT）

- 使用 `transformers` 加载 `bert-base-uncased`
- 提取目标词在句中的上下文向量（支持 WordPiece，按目标词对应 token 平均池化）

4. 对比验证

- 提取两句中目标词向量
- 计算并输出余弦相似度

另外：

- 已加第二个标签页占位（SRL），便于你下一模块直接续写。
- 已在代码中自动下载 NLTK 所需词典数据（wordnet, omw-1.4）。

运行方式：

1. 安装依赖

```
pip install -r requirements.txt
```

2. 启动应用

```
streamlit run app.py
```

- 模块 2：语义角色标注 (SRL) 提取与可视化（对应谓词-论元结构与基于依存关系的 SRL）——在第二个标签页中，实现语义角色标注 (SRL) 功能。

1. 提供一个句子输入框，默认文本为：'Apple is manufacturing new smartphones in China this year.'

2. 轻量级近似实现：由于训练完整的端到端 SRL 模型较重，请使用 `spaCy` (加载 `en_core_web_sm` 模型) 进行依存句法分析，通过编写启发式规则来近似提取谓词和论元。

- 将句子中的主要动词（ROOT 或 VERB）作为谓词 (Predicate)。
- 将 `nsubj` (主语) 映射为 施事者 A0 (Agent)。
- 将 `dobj` (直接宾语) 映射为 受事者 A1 (Patient)。
- 尝试提取时间修饰语或地点修饰语（如识别介词短语 `prep+pobj` 或时间实体）映射为 AM-LOC 或 AM-TMP。

3. 结构化展示：将提取出的 [A0, 谓词, A1, AM-LOC, AM-TMP] 结构以表格的形式清晰地展示在页面上。

4. 依存图辅助：调用 spacy 的 displaCy，将句子的依存关系图渲染在页面底部，帮助验证提取结果。

Copilot & GPT-5.2-Codex:

已完成模块 2，并把 SRL 功能接到了第二个标签页里，满足你列的 4 个要求。

本次实现内容：

1. 输入框

- 在 SRL 标签页新增句子输入框，默认值为 `Apple is manufacturing new smartphones in China this year.`

2. 启发式 SRL 提取 (spaCy 依存分析)

- 谓词 `Predicate`：优先取 `ROOT` 且词性为 `VERB/AUX`，否则回退到第一个 `VERB`
- `A0 (Agent)`：`nsubj / nsubjpass`
- `A1 (Patient)`：`dobj / obj / attr`
- `AM-LOC / AM-TMP`：结合 `prep + pobj` 与实体类型 (`GPE/LOC/FAC`、`DATE/TIME`) 及时间副词规则提取

3. 结构化展示

- 按 `[A0, Predicate, A1, AM-LOC, AM-TMP]` 输出为表格

4. 依存图可视化

- 用 displaCy 渲染依存句法图并展示在页面底部 (HTML 嵌入)

补充说明：

- 我还加了运行时保护：如果本地没装 spaCy 或没下载模型，页面会提示安装命令，不会直接崩溃。
- 你需要先安装并下载模型后再运行 SRL：
 - `pip install -r requirements.txt`
 - `python -m spacy download en_core_web_sm`

2. 实验总结

2.1 模块 1 观察任务：1. 观察 Lesk 算法（基于词典重合度）能否准确区分金融“bank”和河岸“bank”。2. 观察 BERT 输出的余弦相似度。即使都是 “bank” 这个词，在不同语境下它们的向量相似度是否较低？这验证了课件 P10 提到的“动态上下文向量”概念。

- 从实验结果看，Lesk 在这组例子里基本能把两种 “bank” 区分开：在 “deposit my money” 的语境中更倾向金融机构义，而在 “river bank” 的语境中会落到“河岸”义，这说明基于释义重合的传统方法在上下文线索足够明确时依然有效。与此同时，BERT 对两句中同一词的向量相似度通常不会很高，反映出模型并不是给 “bank” 一

个固定表示，而是会根据周围词语动态调整语义表征。也就是说，词面相同不等于语义相同，这正是动态上下文向量最直观的体现。

2.2 模块 2 观察任务：输入课件 P43 的经典结构句子（替换为英文，如 "Chengfei Company is manufacturing civil aircrafts"）。观察程序是否能准确提取出 A0（施事）和 A1（受事）。对比页面底部的依存树，理解 P49 提到的“论元与谓词的语法关系体现为依存关系”。

- 从这组测试句子看，程序对核心语义角色的抽取整体是可信的：在 “Chengfei Company is manufacturing civil aircrafts” 这类典型主谓宾结构中，A0 能稳定对应施事者 “Chengfei Company”，A1 也能较准确落到受事者 “civil aircrafts”。结合底部依存树可以更直观地看到这种结果并非猜测，而是直接来自句法关系本身，例如主语与谓词的依存连接、宾语与动词的支配关系。也就是说，SRL 中谁做了什么这件事，确实可以通过依存结构被清晰地表达出来，这和课堂里论元与谓词关系由依存关系承载的观点是一致的。